

INFO 0702 CM2

Isolation

Emilien Bourdy-Liébart



**UNIVERSITÉ
DE REIMS
CHAMPAGNE-ARDENNE**

8 octobre 2025

Sommaire

- 1 Isolation
 - Théorie
 - Pratique

Qu'est-ce que c'est ?

- Noyau permet l'existence de plusieurs instances d'espaces utilisateurs isolées
 - conteneurs (LXC, Docker, Podman)
 - zones (Solaris containers)
 - serveurs privés virtuels (OpenVZ)
 - partitions
 - environnements virtuels
 - noyaux virtuels (DragonFly BSD)
 - prisons (FreeBSD jail, chroot jail)
- « Vrais » ordinateurs du point de vue des programmes qui y sont exécutés.
 - Programmes ont accès à toutes les ressources d'un ordinateur
 - Dans un conteneur, accès restreint au contenu du conteneur

Isolateur I

- Mécanisme système assurant l'indépendance de plusieurs exécutions
- Implique la gestion d'environnement séparés
- Mise en place d'un *super scheduler*
- Protection des ressources des éléments isolés
- Nécessite de définir
 - la base commune entre les environnements isolés
 - mais aussi avec l'*host*
 - Les éléments spécifiques à chaque système isolé
 - Les éléments spécifiques à l'*host*
- Différents degrés d'isolation

Isolateur II

Exemple (`chroot`)

- *change root*
- Plus ancienne solution (1979)
- Isolation de ressources disque
- Outil GNU élément des *coreutils*
- Isolation par changement de racine
- Permet de limiter un processus à un ensemble de répertoire
- Étendu par les BSD *jails*

Isolateur III

Exemple (Linux-VServer)

- Isolation de contexte de sécurité
- Combiné à du `chroot`
- Correctif du noyau Linux
 - nécessite de recompiler le noyau
 - Debian proposait une distribution avec (dernière version = 6.0 'Squeeze')
- Définition d'espaces utilisateurs
- Partage du noyau entre les VServer et l'*host*
- Utilisable pour exécuter des serveurs

Outils pour l'isolation

Namespace Linux

- isolation des éléments systèmes
 - Unix Time-Sharing
 - PID
 - Réseau
 - Utilisateur
 - Montage
- `cgroups`
- IPC
- *Time*

Namespace I

- Conteneur est une notion inconnue du noyau
 - Outils haut niveau parlent de conteneur
 - Au niveau noyau $\rightarrow$ *namespaces*
 - Les *namespaces* ne sont pas des conteneurs
- $\Rightarrow$ Une brique utilisée pour construire des conteneurs

Namespace II

Pourquoi ? (1/2)

- Beaucoup de notions uniques ou de valeurs communes dans tout le système Unix
- Tous les processus voient la même chose et y accèdent de la même façon
 - *hostname* et le même pour tout le monde
 - Changer le *hostname* « brutalement » peut casser le système...
 - chaque processus a un PID unique
 - configuration réseau commune à tous les processus du système
 - tout le monde voit les même montages sur les mêmes répertoires

Namespace III

Pourquoi ? (2/2)

- À la création d'un conteneur, l'unicité de ces notions empêchent le *guest* de fonctionner correctement :
 - *guest* ne peut pas changer son *hostname* sans modifier celui de l'*host*
 - processus initial du *guest* n'aura pas le PID 1
 - la plupart des système de démarrage (sysV, systemd) refusent de démarrer s'ils ne sont pas PID 1
 - s'ils ne sont pas PID 1, il ne recevront pas les processus orphelins
 - le *guest* ne peut pas changer sa configuration réseau sans modifier l'*host*
 - si l'*host* utilise un port réseau, le *guest* ne pourra pas l'ouvrir

Namespace Unix Time-Sharing I

Comment ça fonctionne ?

Exemple (*hostname*)

- *hostname* est le « nom » de la machine
- obtenu avec `hostname`
- peut être modifié avec `hostname <new_name>` (nécessite d'être `root`)
- Par défaut tous les processus voient le même *hostname*
- Si un processus le modifie, il est modifié pour tous les processus
- Contournement
 - ❶ Création d'un *namespace* de type UTS (Unix Time-Sharing)
 - ❷ On met nos processus à l'intérieur

Namespace Unix Time-Sharing II

⇒ Un *namespace* est un regroupement de processus qui ont une vision commune d'une ressource partagée.

- Plusieurs types de *namespaces* correspondant aux ressources à isoler
- UTS regroupent un certain nombre de variables globales du noyau
 - *hostname*
 - espaces de nommages des socket unix
 - sémaphores posix
 - ...

Namespace Unix Time-Sharing III

Exemple (Création d'un *namespace* UTS)

```
root:~# hostname  
original  
root:~# unshare --uts  
root:~# hostname  
original  
root:~# hostname newname  
root:~# hostname  
newname
```

Dans un autre terminal

```
root:~# hostname  
original
```

Namespace Unix Time-Sharing IV

Exemple (Rejoindre un *namespace*)

```
root:~# hostname  
newname  
root:~# echo $BASHPID  
500384
```

Dans un autre terminal

```
root:~# hostname  
original  
root:~# nsenter -a -t 500384  
root:~# hostname  
newname
```

Namespace Unix Time-Sharing V

- **systemd** récents :
 - **hostname** n'est plus fourni par **systemd** → **inetutils**
- Par défaut
 - Tous les enfants sont créés dans le même *namespace* que leur parents
 - Les *namespaces* disparaissent lorsque leur dernier utilisateur se termine

Pour connaître tous les *namespaces* sur notre système → **lsns**

Colonne	Valeur
NS	Identifiant unique du <i>namespace</i>
TYPE	Type du <i>namespace</i>
NPROCS	Nombre de processus contenus dans le namespace
PID	PID du processus qui a créé le <i>namespace</i>
USER	Utilisateur qui a créé le processus
COMMAND	Commande du processus créateur

Namespace Unix Time-Sharing VI

```
root:~# lsns -t uts
```

NS	TYPE	NPROCS	PID	USER	COMMAND
4026531838	uts	299	1	root	/lib/systemd/systemd -- system --deserialize 60
4026532233	uts	1	366509	root	- /lib/systemd/systemd- udevd
4026532325	uts	1	189196	systemd-timesync	- /lib/systemd/systemd- timesyncd
4026532380	uts	1	701	root	- /lib/systemd/systemd- logind
4026532579	uts	2	500384	root	-bash
4026532603	uts	1	436270	root	/lib/systemd/systemd- machined

Namespace Unix Time-Sharing VII

Quelques remarques sur `lsns`

- `lsns` obtient ses informations de `/proc/pid/ns/`
- Seul `root` y a accès
- Certains *namespaces* limitent la visibilité des *namespaces* eux-mêmes
- Dans un conteneur, seuls les *namespaces* créés par ce conteneur sont visibles
- Depuis l'*host*, tous les *namespaces* sont visibles, y compris ceux du conteneur

Namespace Unix Time-Sharing VIII

Exemple (Différence root/user)

```
info0702:~$ lsns -t uts
```

	NS	TYPE	NPROCS	PID	USER	COMMAND
4026531838	uts		299	1	info0702	xfce4-session

```
info0702:~$ sudo !!
```

	NS	TYPE	NPROCS	PID	USER	COMMAND
4026531838	uts		166	1	root	/sbin/init
4026532181	uts		4	302	root	- /usr/lib/systemd/systemd-userdbd
4026532183	uts		1	318	root	- /usr/lib/systemd/systemd-udevd
4026532249	uts		1	392	root	- /usr/lib/systemd/systemd-logind
4026532420	uts		1	1037	polkitd	- /usr/lib/polkit-1-polkitd
4026532250	uts		1	1878	root	-bash
4026532255	uts		2	2101	info0702	bwrap --unshare-all ...

Namespace pour les PID I

Rappel sur les PID (1/2)

- Processus IDentifier
- Identifiant numérique unique à chaque processus
- Fourni par le noyau à la création du processus
- Jamais modifié dans la vie du processus
- Utilisés dans de nombreux appels systèmes pour indiquer au noyau sur quel processus nous voulons agir (`kill`)

Namespace pour les PID II

Rappel sur les PID (2/2)

- Cas particulier : PID 1
 - Seul processus créé spontanément par le noyau au démarrage une fois initialisé
 - PID 1 est chargé du démarrage de tous les autres processus du système
 - Si un processus est orphelin, il est réaffecté à PID 1
- ⇒ PID 1 doit savoir gérer les **SIGCHLD** de processus qu'il n'a pas lui-même créé
- PID 1 n'a pas le droit de se terminer ⇒ **kernel-panic**
 - Si un processus fait l'appel système **reboot()**, le noyau envoie un signal **SIGTERM** au PID 1

Dans le cas d'un conteneur, on a besoin de démarrer le PID 1 qui existe déjà sur l'*host*.

Namespace pour les PID III

Création d'un *namespace* de PID

```
root:~# unshare --pid --fork --mount-proc
root:~# echo $BASHPID
1
root:~# PID=`bash -c "echo $BASHPID"`
root:~# echo $PID
4
```

- `--fork` force `unshare` à créer un nouveau processus au lieu de réutiliser le sien
- Sinon notre `bash` n'aurait pas son PID dans le nouveau *namespace*
- `--mount-proc` demande à `unshare` de faire le nécessaire pour que `/proc` soit cohérent avec le *namespace* de PID

Namespace pour les PID IV

Remarques sur l'exemple précédent

- C'est bien **bash** qui est notre PID 1
 - Les orphelins seront reparentés à **bash**
 - Si **bash** se termine tous les processus de ce *namespace* se termine
 - **reboot()** fait appel au PID 1.
- attention, **reboot()** fait appel au **systemd** global ⇒ ça ne redémarre pas le *namespace* mais le système entier

Pour vérifier les PID avec **lsns** dans le nouveau *namespace* :

```
root:~# lsns --type pid
```

	NS	TYPE	NPROCS	PID	USER	COMMAND
4026532580	pid		2	1	root	-bash

Namespace pour les PID V

Et dans un autre terminal :

```
root:~# lsns --type pid
```

	NS	TYPE	NPROCS	PID	USER	COMMAND
4026531836	pid		258	1	root	/lib/systemd/systemd
--system --deserialize 60						
4026532288	pid		54	426124	foo	/opt/google/chrome/ chrome --type=zygote
4026532514	pid		1	426127	foo	/opt/google/chrome/ nacl_helper
4026532580	pid		1	556909	root	-bash

Namespace pour les PID VI

Quelques remarques sur l'exemple précédent

- Sur l'*host*, PID 1 est bien celui de base
- Chrome crée des *namespaces* de PID pour protéger ses onglets
- Notre nouveau *namespace* est présent, mais n'a pas la valeur 1

⇒ PID différent si l'on regarde à l'intérieur ou à l'extérieur du *namespace*

Namespace pour les PID VII

Unicité des PID

- Les processus du *namespace* ont deux PID :
 - ❶ le « faux » visible de l'intérieur
 - ❷ le « vrai » visible de l'extérieur
- Un **kill** sur le **bash** ne prendra pas le même PID depuis l'intérieur ou l'extérieur du *namespace*
- C'est au noyau de s'occuper de garder la cohérence

Namespace réseau I

Qu'est-ce que c'est ?

- Regroupe des processus qui ont leur propre pile réseau
- Isolée de la pile réseau du reste du système
- Elles ont :
 - leur propres interfaces réseau
 - leurs propres règles **iptables**
 - leurs propre boucle locale isolée du reste du système

Namespace réseau II

```
root:~# ip -br link list
lo      UNKNOWN    00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0    UP         8c:ae:4c:f1:49:9c <BROADCAST,MULTICAST,UP,LOWER_UP>
```

```
root:~# unshare --net
root:~# ip -br link list
lo      DOWN       00:00:00:00:00:00 <LOOPBACK>
root:~# ip link set lo up
root:~# ip -br link list
lo      UNKNOWN    00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
```

Namespace réseau III

Remarques sur l'exemple précédent

- ❶ Nous regardons les interfaces réseau disponibles sur l'*host*
 - ❶ la boucle locale
 - ❷ une interface Ethernet standard
 - ❷ Nous entrons dans un *namespace* réseau
 - ❸ La seule interface disponible est `lo` qui est `down`
 - ❹ Nous la mettons à `up` immédiatement
- ⇒ Le *namespace* est isolé, il est impossible de communiquer avec l'extérieur car pas d'interface réseau

Namespace réseau IV

Création d'interfaces

- Façon simple : utiliser les **MACVLAN** :
 - ➊ Nous allons demander à notre carte Ethernet d'écouter une deuxième adresse MAC
 - ➋ Nous allons faire en sorte que les paquets utilisant cette deuxième adresse MAC soient vu comme utilisant une autre interface réseau
 - ➌ Nous allons « débrancher » cette deuxième interface de notre *host* et la « rebrancher » dans notre conteneur
- Notre conteneur aura sa propre interface réseau logiquement indépendante de celle de l'*host* mais qui utilisera physiquement la même carte réseau

Namespace réseau V

Dans le *namespace* d'origine :

```
root:~# ip link add mymacvlan link eth0 type macvlan mode bridge
```

```
root:~# ip -br link list
```

```
lo                UNKNOWN 00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>  
eth0              UP        8c:ae:4c:f1:49:9c <BROADCAST,MULTICAST,UP,  
                  LOWER_UP>  
mymacvlan@eth0    DOWN      32:9c:52:14:73:28 <BROADCAST,MULTICAST>
```

Namespace réseau VI

- Nous avons créé une nouvelle interface réseau
- L'adresse MAC a été créée automatiquement par le noyau
- Elle n'a pas d'adresse IP
- Nous commençons d'abord par la transférer vers notre nouveau *namespace* réseau

Namespace réseau VII

Dans notre *namespace* réseau :

```
root:~# echo $BASHPID  
2186254
```

Dans notre *namespace* d'origine

```
root:~# ip netns attach myns 2186254  
root:~# ip link set mymacvlan netns myns  
root:~# ip -br link list  
lo                UNKNOWN 00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>  
enx8cae4cf1499c UP 8c:ae:4c:f1:49:9c <BROADCAST,MULTICAST,UP,LOWER_UP>
```

Dans notre *namespace* réseau

```
root:~# ip -br link list  
lo                UNKNOWN 00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>  
mymacvlan@if31 DOWN 32:9c:52:14:73:28 <BROADCAST,MULTICAST>  
root:~# dhclient mymacvlan
```


Namespace réseau VIII

Autres façons de partager le réseau

- IPVLAN
 - gestion de deux adresses IP au lieu de deux adresses MAC
 - chaque adresse IP devient une interface réseau
 - inutilisable pour partager une interface entre conteneurs
- VLAN
 - chaque VLAN apparaît comme une interface différente
 - possibilité de transférer ces interfaces vers différents *namespaces*
- VETH
 - création d'une paire d'interfaces virtuelles
 - reliées par un câble virtuel
 - une fois transférées vers des *namespaces*, elles peuvent communiquer entre elles, mais pas vers le réseau en général

Namespace réseau IX

Qui fait quoi ?

- Avec un gestionnaire de conteneur complet, c'est lui qui créer et connecte les interfaces réseau
- Généralement basé sur des *bridges* connectés à des interfaces physiques
- Le gestionnaire peut aussi faire la configuration IP
- Sinon c'est au système d'exploitation *guest* de faire la configuration

User-namespaces I

Qu'est-ce que c'est

Comme pour les *namespaces* de PID donnent une vision différente des PID, les *user-namespaces* donne une vision différentes des UID.

Permet de faire en sorte qu'un processus pense être exécuté en tant que UIDx, mais pour le noyau, il est exécuté en tant que UIDy.

Devient intéressant si $\text{UIDx} = 0 \Rightarrow$ le processus pense être exécuté en tant que **root**.

User-namespaces II

En tant qu'utilisateur normal <user> :

```
user:~$ unshare -r
```

```
root:~# id
```

```
uid=0(root) gid=0(root) groupes=0(root),65534(nogroup)
```

`unshare -r` permet de devenir `root` alors que notre utilisateur est un utilisateur normal

```
root:~# ls -l /home
```

```
total 4
```

```
drwxr-xr-x 43 root root 4096 9 Nov 19:21 user
```

```
root:~# cd /root
```

```
-bash: cd: /root: Permission denied
```

```
root:~# ls -dl /root
```

```
drwx----- 9 nobody nogroup 4096 2 Nov 18:42 /root/
```

User-namespaces III

Remarques sur l'exemple précédents

- Notre compte « **root** » n'a pas le bon comportement :
 - ❶ les fichiers de **user** appartiennent maintenant à **root**
 - ❷ nous n'avons pas le droit d'entrer dans le répertoire **/root**
 - ❸ le répertoire **/root** n'appartient pas à **root**, mais à **nobody**
- Ce qu'il s'est passé :
 - ❶ nous sommes entrés dans un *user-namespace*
 - ❷ le noyau nous indique que nous sommes **root**
 - ❸ en vérité nous sommes toujours **user** tous les UID qui ne sont pas ceux de **user** sont cachés derrière l'identifiant **nobody**
 - ❹ seules les vraies permissions sont utilisées

User-namespaces IV

`unshare -r` permet de configurer le *namespace* pour nous en tant que `root`.
Pour une configuration manuelle, en tant qu'utilisateur normal :

```
user:~$ id
uid=1000(user) gid=1000(user)
```

```
user:~$ unshare --user
nobody:~$ id
uid=65534(nobody) gid=65534(nogroup) groups=65534(nogroup)
```

```
nobody:~$ ls -l /home
total 4
drwx-xr-x 43 nobody nogroup 4096 9 Nov 19:21 user
nobody:~$ ls -ld /root
drwx----- 9 nobody nogroup 4096 2 Nov 18:42 /root/
nobody:~$ cd /root
-bash: cd: Permission denied
nobody:~$ echo $BASHPID
37642
```

User-namespaces V

Depuis un autre *namespace*

```
root:~# echo "0      1000      1" > /proc/37642/uid_map
```

Enfin, dans le nouveau *namespace*

```
nobody:~$ id
```

```
uid=0(root) gid=65534(nogroup) groups=65534(nogroup)
```

User-namespaces VI

Remarques sur l'exemple précédent (1/2)

- Nous commençons dans une situation étrange :
 - Nous sommes **nobody**
 - Tous les fichiers et tous les répertoires appartiennent à **nobody** mais nous ne pouvons pas faire ce que nous voulons
⇒ le noyau a tout masqué
- Nous récupérons le PID de notre **bash**
- En l'écrivant dans l'**uid_map**, nous expliquons au noyau que nous souhaitons mapper les UID dans ce *namespace* :
 - l'UID 0 (intérieur) est mappé à l'UID 1000 (extérieur) et un seul UID est mappé par cette ligne
 - l'écriture doit être faite depuis l'extérieur du *namespace*
 - il est possible d'écrire plusieurs lignes dans ce fichier
 - il n'est possible d'écrire dans ce fichier qu'une seule fois
 - il existe un fichier **gid_map** pour les groupes

User-namespaces VII

Remarques sur l'exemple précédent (2/2)

- Il est donc possible d'avoir tout un espace d'UID non utilisés par l'*host* pour avoir son propre **root** et ses propres utilisateurs
- Nom complet de **unshare -r** est **unshare --map-root-user**

Il est donc possible de prétendre être **root** sans en avoir les pouvoirs.
Comment corriger cela ?

Commençons par créer un *namespace* UTS :

```
user:~$ unshare --uts  
unshare: failed: Operation denied
```

Il faut être **root** pour créer des *namespaces*. Comment faire semblant d'être **root** ?

User-namespaces VIII

```
user:~$ unshare -r
root:~# hostname
original
root:~# hostname newname
hostname: you must be root to change the host name
root:~# unshare --uts
root:~# hostname newname
root:~# hostname
newname
```

User-namespaces IX

Remarques sur l'exemple précédent

Un utilisateur normal a le droit de créer des *user-namespace* et le compte **root** d'un *user-namespace* a le droit de créer des *namespaces* dans son *user-namespace*

- ➊ Nous créons un *user-namespace*
- ➋ Nous ne pouvons pas changer le *hostname* car nous sommes un faux **root**
- ➌ Nous créons un *namespace* UTS dans notre *user-namespace*
- ➍ Nous pouvons désormais changer le *hostname* qui ne sera vu que dans ce *namespace* UTS

L'empilement de *namespace* est l'une des clés du fonctionnement des conteneurs.

Attention, les règles complètes sur l'utilisation des *user-namespace* sont bien plus complexes que décrites ici. Un tel système serait trop vulnérable. cf. `user_namespace(7)`

Mount-namespace I

Qu'est-ce que c'est ?

Les *mount-namespaces* permettent d'isoler la vision qu'ont les processus des montages et donc, de changer complètement l'arborescence de fichiers que nos processus peuvent voir.

Mount-namespace II

Exemple (Création d'un *mount-namespace*)

```
user:~$ unshare -r --mount
root:~# mkdir mymountpoint
root:~# mount -t tmpfs none mymountpoint
root:~# mount
[...]
none on /home/user/mymountpoint type tmpfs (rw,
relatime,uid=1000,gid=1000)
```

Depuis un autre terminal :

```
user:~$ mount
[...]
user:~$
```

mymountpoint n'apparaît pas dans l'autre terminal.

Mount-namespace III

Remarques sur l'exemple précédent

- ❶ Nous créons un *mount-namespace*
 - ❷ Nous montons un système de fichier dans ce *namespace*
 - ❸ Ce montage n'est pas visible de l'extérieur
- ⇒ la vision des montages n'est pas partagée

Mount-namespace IV

Particularités des *mount-namespaces* (1/2)

- Ils sont généralement utilisés avec **chroot**
- Les *bind-mounts* permettent d'utiliser un répertoire comme source d'un montage $\Rightarrow$ le contenu du répertoire sera donc visible à deux endroits
- En montant un répertoire extérieur à l'intérieur d'un **chroot**, nous avons un répertoire partagé
- Les *bind-mounts* permettent de changer les propriétés des montages pour un sous-répertoire :
 - on peut monter un répertoire par-dessus lui-même. Il devient *read-only*
 - on peut ensuite monter un sous-répertoire de ce répertoire par-dessus lui-même pour le remettre *read-write*

Mount-namespace V

Particularités des *mount-namespaces* (2/2)

- Ces surcouches permettent de monter un répertoire (généralement vide) par-dessus un autre répertoire source :
 - lorsqu'un fichier est disponible dans les deux répertoires, c'est la version « du dessus » qui sera utilisée
 - si un fichier est modifié, la nouvelle version sera sauvegardée dans le répertoire « du dessous »
- ⇒ on peut ainsi avoir plusieurs conteneurs utilisant la même arborescence de fichiers mais en ayant des copies différentes des données modifiées

Mount-namespace VI

Attention, tout ce que **root** peut faire, **root** peut le défaire. Tous les montages réalisés pour protéger des données ou un système ne servent à rien si un processus dangereux peut le démonter avec **umount** :

```
user:~$ mkdir mymountpoint
user:~$ touch mymountpoint/fichiersecret.txt

user:~$ unshare -r --mount
root:~# mount -t tmpfs none mymountpoint
root:~# ls mymountpoint
# Rien de visible
root:~# unshare -r
root:~# mount
none on /home/user/mymountpoint type tmpfs (rw,
relatime,uid=1000,gid=1000)
root:~# umount mymountpoint
umount: /home/user/mymountpoint: must be superuser to unmount.
```

Mount-namespace VII

En créant un deuxième *user-namespace* après avoir fait nos montages, nous changeons de *user-namespace* et notre nouveau faux **root** n'est plus le bon faux **root** et ne peut plus avoir d'effet sur les montages du *mount-namespace*.

La technique des couches de *user-namespaces* est la base de l'isolation des conteneurs :

- ❶ on crée un premier *user-namespace* pour pouvoir avoir les permissions de configuration
- ❷ on crée les *namespaces* secondaires et on positionne toutes les configurations nécessaires
- ❸ on crée un deuxième *user-namespace* pour protéger nos configurations
- ❹ on crée un deuxième jeu de *namespaces* secondaires que le conteneur pourra administrer lui-même

Autres *namespaces*

- **cgroup**
 - Limiter la vision des processus sur les *control-groups*
 - Les processus ne peuvent donc pas savoir que certaines ressources leur sont interdites
- **IPC**
 - Limiter l'accès à certaines ressources partagées
 - files de messages
 - sémaphores
 - mémoires partagées
 - sockets anonymes
- **Time**
 - isolation de l'heure
 - Permet de changer l'heure du système dans un *namespace* sans affecter l'*host*

Opérations possibles des conteneurs

- Sur l'*host*
 - Programme peut voir :
 - capacités matérielles
 - données pouvant être lues ou écrites
 - périphériques connectés
- Dans un conteneur, le système d'exploitation peut :
 - Autoriser / restreindre des ressources en fonction
 - du programme qui les demande
 - du compte utilisateur
 - Masquer des ressources
- Attention, ce n'est pas parce qu'un programme dans un conteneur ne voit pas une ressource qu'il n'a pas interagi avec

Utilisations de conteneurs I

- Couramment utilisés pour l'hébergement virtuel
 - allocation en toute sécurité des ressources matérielles entre un grand nombre d'utilisateurs qui ne peuvent pas nécessairement se faire confiance
 - meilleur rendement du serveur
- Indépendance du matériel et de sa gestion de ressources supplémentaires
- Sécurité améliorée par le cloisonnement
- Équilibrage des charges dynamique des conteneurs entre les nœuds d'un cluster

Utilisations de conteneurs II

Utilisation des ressources

- Nécessite moins de ressources que la virtualisation complète

Utilisations de conteneurs II

Utilisation des ressources

- Nécessite moins de ressources que la virtualisation complète
- > partitions virtuels utilisent l'API système normale du système d'exploitation
 - pas besoin d'être soumis à une émulation
 - ni d'être exécuté dans une machine virtuelle intermédiaire
 - pas de support matériel pour avoir des performances efficaces

Utilisations de conteneurs III

Souplesse d'un conteneur

- Moins flexible que de la virtualisation complète

Utilisations de conteneurs III

Souplesse d'un conteneur

- Moins flexible que de la virtualisation complète

- système d'exploitation exécuté directement avec les appels systèmes
 - impossible de lancer un système d'exploitation différent de l'*host*
 - ou de lancer un noyau *host* différent

Utilisations de conteneurs IV

Stockage

- Mécanisme de copie sur écriture au niveau des fichiers
- Système de fichiers partagé entre les partitions
- Copie automatique en local des fichiers lors de l'écriture

Questions I

Exercice (Définitions)

- 1 Qu'est-ce qu'un isolateur
- 2 Quel est le plus ancien des isolateurs ?
- 3 Quel est le nom donné aux isolateurs dans des systèmes comme Docker, LXC ou encore Podman ?
- 4 À quoi ont accès les programmes à l'intérieur d'un isolateur ?
- 5 Les programmes exécutés dans un isolateur savent-ils qu'ils sont dedans ?
- 6 Que ne peut pas faire un isolateur ?

Questions II

Exercice (*Namespaces*)

- ❶ Citer 4 des 8 types de *namespaces* évoqué jusqu'à maintenant
- ❷ Qu'est-ce qu'un *namespace*
- ❸ Pourquoi en a-t-on besoin ?
- ❹ Quelle est la commande permettant de créer des *namespaces* ?

Questions III

Exercice (*Namespace* UTS)

- ❶ Que signifie UTS ?
- ❷ Que fait la suite de commandes suivante ?

```
root:~# unshare --uts  
root:~# hostname newname
```

- ❸ Quel est le nom de la commande pour rejoindre un *namespace* UTS ?
- ❹ Quel le nom de la commande permettant d'afficher les *namespaces* visibles dans mon contexte ?

Questions IV

Exercice (*Namespace* des PID)

- ❶ Qu'est-ce qu'un PID ?
- ❷ Qui fournit le PID d'un processus ?
- ❸ Quelles sont les particularités du PID 1 ?
- ❹ Quelle est la commande pour créer un *namespace* de PID ?
- ❺ Si un processus dans mon conteneur fait l'appel système `reboot()`, est-ce que l'*host* redémarre ?
- ❻ Donner un exemple de programme où il est intéressant d'utiliser les *namespaces* de PID.
- ❼ Le PID du programme qui a créé le *namespace* de PID est-il le même dans le conteneur et sur l'*host* ?
- ❽ Qui s'occupe de la cohérence des PID internes et externes ?

Questions V

Exercice (*Namespace* réseau)

- ❶ Qu'est-ce c'est ?
- ❷ Quelle est la commande à utiliser ?
- ❸ Est-il possible de communiquer avec l'extérieur par défaut ?
- ❹ Qu'est-ce
 - ❶ qu'un MACVLAN ?
 - ❷ qu'un IPVLAN ?
 - ❸ qu'un VLAN
 - ❹ qu'un VETH
- ❺ Quelle est la procédure générale de création d'un MACVLAN ?

Questions VI

Exercice (*User-namespaces*)

- ❶ Qu'est qu'un *user-namespace* ?
- ❷ À quoi ça sert ?
- ❸ Quels sont les noms des utilisateurs et groupes cachés par Linux ?
- ❹ Quel UID a **root** ?
- ❺ Quelle est la procédure générale pour créer un *user-namespace* où le faux **root** aurait tous les droits ?

Questions VII

Exercice (Mount-namespaces)

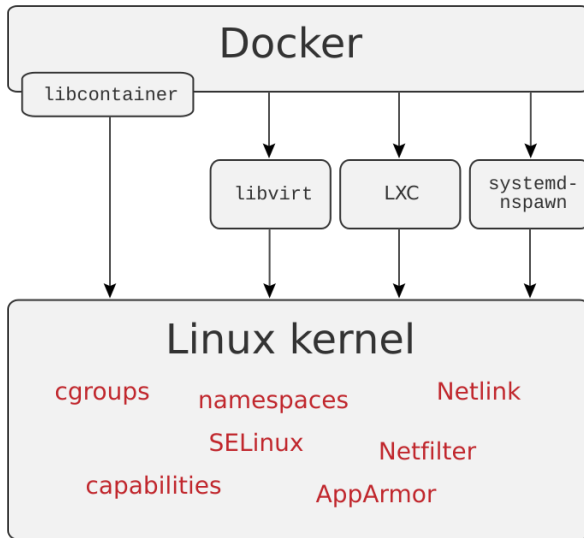
- ❶ Qu'est-ce que c'est ?
- ❷ Si je monte un répertoire dans un conteneur, est-il visible dans les partitions montées sur l'*host* ?
- ❸ Où utilise-t-on les *mount-namespaces* ?
- ❹ Comment rendre un répertoire *read-only* avec les *mount-namespaces* ?
- ❺ Comment le remettre en mode *read-write* ?
- ❻ Qu'est-il dangereux dans le fait que ce soit **root** qui monte ce type de répertoire ?
- ❼ Comment contrer cela ?
- ❽ Donner un avantage de ces surcouches de points de montage.

Docker I

- 2013
- Docker Inc
- Open-source
- Extension de LXC + API haut niveau
- *Stateless* → tout ce qui est fait dans le conteneur est perdu une fois éteint
- Système d'images



Docker II



Installation de Docker I

- Docker Desktop pour MacOS et Windows
- <https://hub.docker.com/>
- Peut nécessiter un compte
- Sur Windows, nécessité d'installer un WSL et de redémarrer
- MacOS :
 - ① En utilisant Docker.dmg à télécharger ou en ligne de commande :

```
$ sudo hdiutil attach Docker.dmg
```

```
$ sudo /Volumes/Docker/Docker.app/Contents/MacOS/install
```

```
$ sudo hdiutil detach /Volumes/Docker
```

Installation de Docker II

Pour Linux, dépend du système.

Pour Debian-*like*, il faut télécharger le `.deb` :

```
$ sudo apt install gnome-terminal  
$ sudo apt update  
$ sudo apt install ./docker-desktop-amd64.deb  
$ systemctl --user enable docker-desktop
```

Pour ArchLinux :

```
$ sudo pacman -Sy docker  
$ sudo systemctl enable docker.service
```

Installation de Docker III

Attention, ici, il faut explicitement utiliser `docker.service` pour ne pas lancer `docker.socket` qui ralentit fortement le démarrage.

Il est possible de s'ajouter au groupe `docker` pour utiliser Docker sans être `root`.

Installation de Docker IV

Commandes de base

- Télécharger une image → `docker pull`
- Lancer une image → `docker run`
- Voir les images en cours d'exécution → `docker ps`
- Lister les images → `docker ls`
- Manipuler les volumes → `docker volume`
- Manipuler le réseau → `docker network`
- Supprimer des images → `docker rmi` et `docker rm`
- Mettre une image en ligne → `docker push`

Installation de Docker V

Manipulation de base :

```
$ docker run hello-world  
$ docker images  
$ docker pull hello-world  
$ docker ps  
$ docker ps -a  
$ docker rm hash_id  
$ docker ps -a
```


Installation de Docker VI

Redirection de port et montage de volume :

```
$ docker run -d nginx:latest
$ curl http://localhost
$ docker stop ID
$ docker run -d -p 8080:80 nginx:latest
$ curl http://localhost:8080
$ docker stop ID
$ ls www
$ cat www/index.html
$ docker run -d -p 8080:80 -v $PWD/www:/usr/share/nginx/html
nginx:latest
$ curl http://localhost:8080
$ docker stop ID
$ docker system prune
```

Images Docker I

- Organisation en couches
 - Permet de les mettre en cache
 - On ne reconstruit que les couches modifiées
- ⇒ facilite la mise à jour et la maintenance des images
- ⇒ améliore l'isolation
- Utilisation d'un **Dockerfile**

Dockerfile

- Fichier texte
- Deux types d'instructions
 - 1 instructions de construction
 - 2 instructions de métadonnées

Images Docker II

Instruction	Description	Exemple
FROM	Indique l'image de base.	FROM ubuntu:24.10
WORKDIR	Indique le répertoire courant	WORKDIR /
USER	Indique l'utilisateur courant.	USER root
COPY/ADD	Permet de copier des fichiers de l' <i>host</i> dans le futur système de fichier	COPY ./src /app
RUN	Exécute une commande.	RUN apt update && apt install -y cowsay dumb-init
ENV	Définir une variable d'environnement accessible à la phase d'exécution	ENV MESSAGE="Kakoo !"

Images Docker III

Instruction	Description	Exemple
ARG	Comme ENV, mais à la phase de construction	
ENTRYPOINT	Exécutable qui sera le point d'entrée du conteneur (PID 1).	ENTRYPOINT ["/usr/bin/dumb-init", "--"]
CMD	Commande à exécuter dans l'ENTRYPOINT.	CMD ["/bin/bash", "-c", "/usr/games/cowsay \$MESSAGE"]
EXPOSE	Permet de déclarer des ports exposables.	EXPOSE 80
VOLUME	Définir des répertoires pouvant servir de point de montage.	VOLUME /data

Images Docker IV

Instruction	Description	Exemple
LABEL	Inscrire des étiquettes comme métadonnées.	<code>LABEL description="Un conteneur d'exemple"</code>

Exemple (Dockerfile)

```
FROM ubuntu:latest

WORKDIR /

RUN apt update && apt install -y cowsay dumb-init

ENV MESSAGE="Kakoo !"

ENTRYPOINT ["/usr/bin/dumb-init", "--"]

CMD ["/bin/bash", "-c", "/usr/games/cowsay $MESSAGE"]
```

Images Docker V

Utilisation du Dockerfile :

```
$ docker build -t cowsay:latest .  
$ docker images  
$ docker run -it --rm cowsay:latest  
$ docker run -it --rm -e 'MESSAGE=Bonjour' cowsay:latest  
# Changer la variable d'environnement MESSAGE dans le  
# Dockerfile  
$ docker build -t cowsay:new .  
$ docker images  
$ docker run -it --rm cowsay:new  
$ docker tag cowsay:new cowsay:v1.0  
$ docker images  
$ docker rmi cowsay:latest  
$ docker images
```

Docker Compose I

- Outil permettant la gestion d'un écosystème pour le conteneur
- Déjà installé sur MacOS et Windows avec Docker Desktop

Sur Debian-*like* :

```
$ sudo curl -L "https://github.com/docker/compose/releases  
/download/2.30.3/docker-compose-$(uname -s)-$(uname -m)"  
-o /usr/bin/docker-compose && sudo chmod +x  
/usr/bin/docker-compose
```

Sur ArchLinux :

```
$ sudo pacman -Sy docker-compose
```

Docker Compose II

Commandes

- Utilisation d'un fichier `compose.yml`
- Télécharger un fichier de configuration →
`docker compose pull`
- Lancer les conteneurs → `docker compose up`
- Afficher le statut d'une stack → `docker compose ps`
- Afficher les logs d'une stack → `docker compose logs`
- Arrêter une stack → `docker compose stop`
- Supprimer les ressources de la stack →
`docker compose down`
- Valider le fichier YAML → `docker compose config`

Docker Compose III

- Nous avons une application Python qui utilise une base de données Redis
- Redis doit tourner à côté de l'application

→ Utilisation du composer :

- ❶ Créer l'application Python
- ❷ Écrire le fichier `requirements.txt`
- ❸ Créer un `Dockerfile`
- ❹ Écrire le fichier `compose.yml`
- ❺ Construire et lancer l'application avec `docker-compose up`

Docker Compose IV

Dockerfile :

```
FROM python:3.10-alpine
```

```
WORKDIR /code
```

```
ENV FLASK_APP=app.py
```

```
ENV FLASK_RUN_HOST=0.0.0.0
```

```
RUN apk add --no-cache gcc musl-dev linux-headers
```

```
COPY requirements.txt requirements.txt
```

```
RUN pip install -r requirements.txt
```

```
EXPOSE 5000
```

```
COPY . .
```

```
CMD ["flask", "run", "--debug"]
```

Docker Compose V

```
compose.yml :  
  
services:  
  web:  
    build: .  
    ports:  
      - "8000:5000"  
  redis:  
    image: "redis:alpine"
```

Démarrer les conteneurs :

```
$ docker compose config  
$ docker compose up  
# Regarder le resultat sur localhost:8000  
$ docker image ls  
$ docker compose down
```

Docker Compose VI

Modification du `compose.yml` :

```
services:
  web:
    build: .
    ports:
      - "8000:5000"
    develop:
      watch:
        - action: sync
          path: .
          target: /code
  redis:
    image: "redis:alpine"
```

Docker Compose VII

On reconstruit l'image :

```
$ docker compose config  
$ docker compose watch  
# Verifier le resultat  
# Changer le message de l'application  
# Regarder le resultat
```

Docker Compose VIII

- Il est tout à fait possible de séparer les services du fichier de configuration
 - ❶ Création d'un fichier YAML (`infra.yml`)
 - ❷ Mettre le service Redis du `compose.yml` dans `infra.yml`
 - ❸ Ajouter `include` dans le `compose.yml`

Fichier `infra.yml` :

```
services:  
  redis:  
    image: "redis:alpine"
```

Docker Compose IX

Fichier `compose.yml` :

```
include:
  - infra.yaml
services:
  web:
    build: .
    ports:
      - "8000:5000"
  develop:
    watch:
      - action: sync
        path: .
        target: /code
```

Docker Swarm I

Qu'est-ce que c'est ?

- Permet de regrouper plusieurs Docker en *cluster* et les gérer de manière centralisée
- Deux types de nœuds
 - ① nœuds managers
 - gèrent l'état global du *cluster*
 - prennent les décisions concernant le déploiement et la gestion des services
 - peuvent exécuter des conteneurs
 - ② nœuds *workers*
 - exécutent les conteneurs
 - sont aux ordres des managers
- Client Docker envoie les instructions aux managers
- Système de certificats TLS pour interagir entre les conteneurs

Docker Swarm II

3 fonctionnalités de base :

① Définition de l'état de l'application

- Fichier de configuration (type Compose)
- Docker Swarm se charge de maintenir cet état

Déclaration description des services, réseaux, volumes

Orchestration orchestration du déploiement et la gestion des services pour correspondre à la déclaration

Auto-réparation En cas de défaillance, Swarm se charge de rétablir l'état souhaité

② Réseau superposé

- Communication sécurisée sur différents hôtes dans le *cluster*
- Réseaux virtuels générés par Docker

Isolation du réseau Chaque couche réseau est décorrélée du réseau physique

Sécurisé Trafic entre conteneurs chiffrés

Simple Possibilité de déplacer des conteneurs sans toucher au réseau

Docker Swarm III

③ *Rolling updates*

- Mise à jour des service sans interruption

Gradualité Les mise à jour déployées par étape

Contrôle On choisit le nombre de conteneurs mis à jour en même temps

Rollback Système de versionnage

Docker Swarm IV

Commandes

- Initialisation d'un *cluster* → `docker swarm init`
- Générer un jeton d'authentification →
`docker swarm join-token worker`
- Rejoindre un *cluster* → `docker swarm join`
`--token <worker-token> <manager-pi>:2377`
- Lister les nœuds du *cluster* → `docker node ls`
(uniquement manager)
- Inspecter un nœud spécifique →
`docker node inspect <node-id>` (uniquement manager)

Docker Swarm V

Exemple (Fichier Compose)

```
services:
  web:
    deploy:
      replicas: 2
    image: 127.0.0.1:5000/stackdemo
    build: .
    ports:
      - "8000:8000"
  redis:
    image: redis:alpine
```

Docker Swarm VI

Déployer une stack

- Déploiement →
`docker stack deploy -c <conf.yml> <stack-name>`
- Lister les stacks déployées → `docker stack ls`
- Rejoindre un *swarm* →
`docker swarm join --token <token> <ip>:<port>`
- Lister les conteneurs en cours d'exécution →
`docker stack ps <stack-name>`
- Supprimer une stack → `docker stack rm <stack-name>`

Docker Swarm VII

Quelques remarques sur les managers

- Toujours en avoir plusieurs
- Système Raft
 - Modification à la majorité
 - Si *quorum* non atteint, aucune modification ne sera faite
- Recommandé d'en avoir un nombre impair ≥ 3
 - Permet de pouvoir perdre quelques nœuds
 - Permet de les partager sur ≥ 3 serveurs physiques différents
- IP statiques
- Ordonnés comme les maisons au Monopoly

Nombre de <i>managers</i>	Nombre de serveurs	Distribution
3	3	1/1/1
5	3	2/2/1
21	4	6/5/5/5
21	5	5/4/4/4/4

Questions I

Exercice (Docker)

❶ Puis-je mettre une image de

- MacOS
- Windows
- Ubuntu
- ArchLinux

dans un Docker ?

❷ Puis-je utiliser Docker sur

- MacOS ?
- Windows ?
- Ubuntu ?
- ArchLinux ?

Questions II

Exercice (Commandes Docker)

- 1 Quelle est la commande pour lancer une image ?
- 2 Quelle est la commande pour télécharger une image ?
- 3 Quelle est la commande pour nettoyer le système ?
- 4 Quelle est la commande pour afficher les images en cours d'exécution ?
- 5 Que fait `docker push` ?

Questions III

Exercice (Images Docker)

- ❶ Quel est le nom du fichier permettant de générer une image ?
- ❷ Quelle est l'instruction pour indiquer l'environnement de base ?
- ❸ Quelle est la différence entre `COPY` et `ADD` ?
- ❹ Vaut-il mieux avoir plein de commande `RUN` plutôt que peu ?
- ❺ Quelle est la différence entre les instructions `ENV` et `ARG` ?
- ❻ Quelle est la commande pour construire une image ?
- ❼ Si je fais une modification dans mon fichier que se passe-t-il à la régénération de l'image ?
- ❽ Comment faire du versionnage d'images ?

Questions IV

Exercice (Docker Compose)

- À quoi ça sert ?
- Quelle est la commande pour lancer les conteneurs ?
- Que fait `docker compose config` ?
- Quel est le format du fichier de configuration ?
- Quel est l'instruction pour séparer les fichiers de configuration ?
- Comment rendre dynamique les modifications apportées aux applications ?

Questions V

Exercice (Docker Swarm)

- À quoi ça sert ?
- Quels sont les deux types de nœuds ?
- Avec qui interagi le client Docker ?
- Combien est-il recommandé d'avoir de managers ?
- Combien est-il recommandé d'avoir de serveurs physiques ?
- Si mon *swarm* comporte 7 managers, à partir de combien de managers en panne je ne pourrai plus faire de modifications ?
- Quelle est la bonne pratique de répartition de mes 7 managers sur 3 serveurs physiques ?